EC3405 – GUI Programming

Lecture 02 - JavaScript

Ms. Yugani Gamlath, Lecturer, Department of Electrical & Information Engineering,
Faculty of Engineering, University of Ruhuna

Content

Part 01:
Introduction, JS Syntax, JS - Placement in HTML File, Data Types, Variables, Variable Scope, Variable Names, Reserved Words

Part 02:
JS Operators, JS if...else Statements, JS Loops , JS Arrays

Part 03:
JS Functions, JS Events, JS Dialog Boxes



PART 01

Introduction
JS Syntax
JS - Placement in HTML File
Data Types
Variables
Variable Scope
Variable Names
Reserved Words

What is JavaScript?

JavaScript is a **dynamic computer programming language.**

It is lightweight and used as a part of web pages, whose implementations allow client-side script to interact with the user and make dynamic pages.

JS Syntax

<script> tag takes 2 important attributes;

Language – This attribute specifies what scripting language you are using. Typically, its value will be JavaScript.

Type – This attribute is what is now recommended to indicate the scripting language in use and its value should be set to "text/javascript".

JavaScript **ignores spaces, tabs, and newlines** that appear in JavaScript programs. **Semicolons are optional.** JavaScript is a **case-sensitive language.**

```
<script language="javascript" type="text/javascript">

        javascript code

</script>
```

```
<html>
   <body>
      <script language = "javascript" type = "text/javascript">

            document.write("Hello World!")

      </script>
   </body>
</html>
```

JS - Placement in HTML File

Most preferred ways to include JavaScript in an HTML file are as follows.

- ✓ **Script in <head>...</head> section.**
- ✓ **Script in <body>...</body> section.**
- ✓ **Script in <body>....</body> and <head>...</head> sections.**
- ✓ **Script in an external file and then include in <head>...</head> section.**

JS in <head>…</head> Section

```
<html>

   <head>

      <script type="text/javascript">

            function sayHello() {
               alert("Hello World")
            }

      </script>

   </head>

   <body>
      <input type="button" onclick="sayHello()" value="Say Hello" />
   </body>

</html>
```

JS in <body>…</body> Section

```
<html>

    <head>
    </head>

    <body>

        <script type="text/javascript">

            document.write("Hello World")

        </script>

        <p>This is web page body </p>

    </body>
</html>
```

JS in <body> and <head> Sections

```
<html>
   <head>
      <script type="text/javascript">

            function sayHello() {
               alert("Hello World")
            }

      </script>
   </head>

   <body>
      <script type="text/javascript">

            document.write("Hello World")

      </script>

      <input type="button" onclick="sayHello()" value="Say Hello" />

   </body>
</html>
```

JS in External File

HTML file (filename.html)

```
<html>
   <head>
      <script type = "text/javascript" src = "filename.js" ></script>
   </head>

   <body>
      Your codes
   </body>
</html>
```

JS file (filename.js)

```
function sayHello() {
   alert("Hello World")
}
```

JS Data Types

JavaScript allows you to work with **3 primitive data types:**

- ✓ **Numbers** Ex: 123, 120.50
- ✓ **Strings** Ex: "This text string"
- ✓ **Boolean** Ex: true or false

JS Variables

Variables can be thought of as **named containers.**

You can place data into these containers and then refer to the data simply by naming the container.

Before you use a variable in a JS program, you must declare it.

Variables are declared with the **var keyword**.

References: https://www.w3schools.com/js/js_variables.asp

https://www.w3schools.com/js/js_let.asp

https://www.w3schools.com/js/js_const.asp

```
<script type = "text/javascript">
      var money;
      var name;
</script>
```

```
<script type = "text/javascript">
      var name = "Ali";
      var money;
      money = 2000.50;
</script>
```

JS Variable Scope

The scope of a variable is the region of your program in which it is defined. JS variables have only 2 scopes.

- ✓ **Global Variables – A global variable has global scope which means it can be defined anywhere in your JavaScript code.**

- ✓ **Local Variables – A local variable will be visible only within a function where it is defined.**

Function parameters are always local to that function. Within the body of a function, a local variable takes precedence over a global variable with the same name. If you declare a local variable or function parameter with the same name as a global variable, you effectively hide the global variable.

JS Variable Scope

```
<html>
   <body onload = checkscope();>
      <script type = "text/javascript">

            var myVar = "global";      // Declare a global variable
            function checkscope( ) {
               var myVar = "local";    // Declare a local variable
               document.write(myVar);
            }

      </script>
   </body>
</html>
```

JS Variable Names

- ✓ You should not use any of the JS reserved keywords as a variable name.
- ✓ JS variable names should not start with a numeral (0-9).
- ✓ They must begin with a letter or an underscore character.
- ✓ JavaScript variable names are case-sensitive.

JS Reserved Words

abstract	else	instanceof	switch
boolean	enum	int	synchronized
break	export	interface	this
byte	extends	long	throw
case	false	native	throws
catch	final	new	transient
char	finally	null	true
class	float	package	try
const	for	private	typeof
continue	function	protected	var
debugger	goto	public	void
default	if	return	volatile
delete	implements	short	while
do	import	static	with

Part 02

JS Operators
JS if...else Statements
JS Loops
JS Arrays

JS Operators

JS supports the following types of operators.

- **Arithmetic Operators**
- **Comparison Operators**
- **Logical Operators**
- **Assignment Operators**
- **Conditional Operators**

JS Operators

Arithmetic Operators

Assume variable A holds 10 and variable B holds 20

Operator	Description
+ (Addition)	**Ex:** A + B will give 30
- (Subtraction)	**Ex:** A - B will give -10
* (Multiplication)	**Ex:** A * B will give 200
/ (Division)	**Ex:** B / A will give 2
% (Modulus)	**Ex:** B % A will give 0
++ (Increment)	**Ex:** A++ will give 11
-- (Decrement)	**Ex:** A-- will give 9

JS Operators

Comparison Operators

Assume variable A holds 10 and variable B holds 20

Operator	Description
= = (Equal)	**Ex:** (A == B) is not true.
!= (Not Equal)	**Ex:** (A != B) is true.
> (Greater than)	**Ex:** (A > B) is not true.
< (Less than)	**Ex:** (A < B) is true.
>= (Greater than or Equal to)	**Ex:** (A >= B) is not true.
<= (Less than or Equal to)	**Ex:** (A <= B) is true.

JS Operators

Logical Operators

Operator	Meaning	Example	Result
&&	Logical and	(5<2)&&(5>3)	False
||	Logical or	(5<2)||(5>3)	True
!	Logical not	!(5<2)	True

JS Operators

Assignment Operators

Operator	Description
= (Simple Assignment)	**Ex:** C = A + B will assign the value of A + B into C
+= (Add and Assignment)	**Ex:** C += A is equivalent to C = C + A
–= (Subtract and Assignment)	**Ex:** C -= A is equivalent to C = C - A
*= (Multiply and Assignment)	**Ex:** C *= A is equivalent to C = C * A
/= (Divide and Assignment)	**Ex:** C /= A is equivalent to C = C / A
%= (Modules and Assignment)	**Ex:** C %= A is equivalent to C = C % A

JS Operators

Conditional Operators

Operator	Description
? : (Conditional)	If Condition is true? Then value X : Otherwise value Y

Syntax	Example
(condition) ? x : y	(z < 18) ? x : y

JS if..else statements

JavaScript supports the following forms of if..else statements.

- **if statement**
- **if...else statement**
- **if...else if... statement**

JS if..else statements

JS if statement

The if statement is the fundamental control statement that allows JS to make decisions and execute statements conditionally.

```
if (expression) {
    Statement(s) to be executed if expression is
true
}
```

Example

Make a "Good day" greeting if the hour is less than 18:00:

```
if (hour < 18) {
  greeting = "Good day";
}
```

The result of greeting will be:

```
Good day
```

JS if..else statements

if...else statement

The 'if...else' statement is the next form of control statement that allows JS to execute statements in a more controlled way.

```
if (expression) {
    Statement(s) to be executed if expression is
true
} else {
    Statement(s) to be executed if expression is
false
}
```

Example

If the hour is less than 18, create a "Good day" greeting, otherwise "Good evening":

```
if (hour < 18) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
```

The result of greeting will be:

```
Good day
```

JS if..else statements

if...else if... statement

The if...else if... statement is an advanced form of if...else that allows JS to make a correct decision out of several conditions.

```
if (expression 1) {
    Statement(s) to be executed if expression 1 is true
} else if (expression 2) {
    Statement(s) to be executed if expression 2 is true
} else if (expression 3) {
    Statement(s) to be executed if expression 3 is true
} else {
    Statement(s) to be executed if no expression is true
}
```

Example

If time is less than 10:00, create a "Good morning" greeting, if not, but time is less than 20:00, create a "Good day" greeting, otherwise a "Good evening":

```
if (time < 10) {
  greeting = "Good morning";
} else if (time < 20) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
```

The result of greeting will be:

```
Good day
```

JS Loops

JS while Loop

The purpose of a while loop is to execute a statement or code block repeatedly as long as an expression is true. Once the expression becomes false, the loop terminates.

```
while (expression) {
    Statement(s) to be executed if expression is true
}
```

Example

```
while (i < 10) {
  text += "The number is " + i;
  i++;
}
```

JS Loops

JS do...while Loop

The do...while loop is similar to the while loop except that the condition check happens at the end of the loop. This means that the loop will always be executed at least once, even if the condition is false.

```
do {
    Statement(s) to be executed;
} while (expression);
```

Example

```
do {
  text += "The number is " + i;
  i++;
}
while (i < 10);
```

JS Loops

JS For Loop

The 'for' loop is the most compact form of looping. It includes the following 3 important parts.

1. The **loop initialization** where we initialize our counter to a starting value. The initialization statement is executed before the loop begins.
2. The **test statement** which will test if a given condition is true or not. If the condition is true, then the code given inside the loop will be executed, otherwise the control will come out of the loop.
3. The **iteration statement** where you can increase or decrease your counter.

```
for (initialization; test condition; iteration statement) {
    Statement(s) to be executed if test condition is true
}
```

Example

```
for (let i = 0; i < 5; i++) {
  text += "The number is " + i + "<br>";
}
```

Activities

- Write a JavaScript code to print numbers from 1 to 10 using for loop.
- Write a JavaScript program that calculates the sum of numbers from 1 to 10 using for loop.
- Write a JavaScript program that prints the numbers from 1 to 5 using a while loop.
- Write a JavaScript program that calculates the sum of numbers from 1 to 5 using a while loop.

JS Arrays

JS arrays are used to store multiple values in a single variable.

An array is a special variable, which can hold more than one value at a time.



Spaces and line breaks are not important. A declaration can span multiple lines:

```
var cars = [
  "Saab",
  "Volvo",
  "BMW"
];
```

JS Arrays

Access the Elements of an Array

You access an array element by referring to the index number.

This statement accesses the value of the first element in cars:

var name = cars[0];

Note: Array indexes start with 0. [0] is the first element. [1] is the second element.

```
var cars = ["Saab", "Volvo", "BMW"];
document.getElementById("demo").innerHTML = cars[0];
```

JS Arrays

Access the Full Array

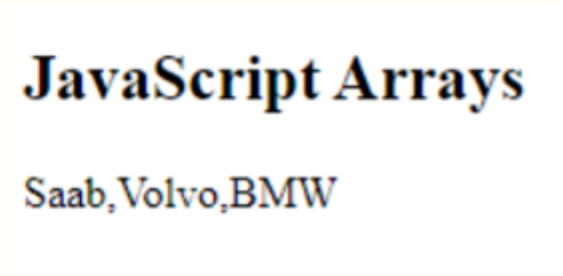
```
<html>
<body>

<h2>JavaScript Arrays</h2>

<p id="demo"></p>

<script>
var cars = ["Saab", "Volvo", "BMW"];
document.getElementById("demo").innerHTML = cars;
</script>

</body>
</html>
```



JS Arrays

Array Properties and Methods

The real strength of JS arrays are the built-in array properties and methods:

```
var x = cars.length;    // The length property returns the number of
elements
var y = cars.sort();    // The sort() method sorts arrays
```

JS Arrays

Length Property

Length property of an array **returns the length of an array** (the number of array elements).

```
<body>
<p id="demo"></p>
<script>
var fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo").innerHTML = fruits.length;
</script>
</body>
```

JS Arrays

Accessing the First Array Element

```
<body>
<p id="demo"></p>
<script>
var fruits = ["Banana", "Orange", "Apple", "Mango"];
var first = fruits[0];
document.getElementById("demo").innerHTML = first;
</script>
</body>
```

JS Arrays

Accessing the Last Array Element

```
<body>
<p id="demo"></p>
<script>
var fruits = ["Banana", "Orange", "Apple", "Mango"];
var last = fruits[fruits.length-1];
document.getElementById("demo").innerHTML = last;
</script>
</body>
```

JS Arrays

Adding Array Elements

The easiest way to add a new element to an array is using the **push()** method:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];
fruits.push("Lemon");      //  adds a new element (Lemon) to fruits
```

New element can also be added to an array using the length property:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];
fruits[fruits.length]  = "Lemon";      //  adds a new element (Lemon) to
fruits
```

JS Arrays

Changing Elements

Array elements are accessed using their index number.

```
<body>

<p id="demo1"></p>
<p id="demo2"></p>

<script>
var fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo1").innerHTML = fruits;
fruits[0] = "Kiwi";
document.getElementById("demo2").innerHTML = fruits;
</script>

</body>
```

Banana,Orange,Apple,Mango

Kiwi,Orange,Apple,Mango

JS Arrays

Deleting Elements

```
<body>

<p id="demo1"></p>
<p id="demo2"></p>

<script>
var fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo1").innerHTML =
"The first fruit is: " + fruits[0];
delete fruits[0];
document.getElementById("demo2").innerHTML =
"The first fruit is: " + fruits[0];
</script>

</body>
```

The first fruit is: Banana

The first fruit is: undefined

Part 03

JS Functions
JS Events
JS Dialog Boxes

JS Functions

A function is a group of reusable code which can be called anywhere in your program.

This eliminates the need of writing the same code again and again.

A JS function is a block of code designed to perform a particular task.

```
function name(parameter1, parameter2, parameter3) {
  //  code to be executed
}
```

JS Functions

Sample Function

```
<html>
<body>

<h2>JavaScript Functions</h2>

<p>This example calls a function which performs a calculation, and returns the
    result:</p>

<p id="demo"></p>

<script>
function myFunction(p1, p2) {
    return p1 * p2;
}
document.getElementById("demo").innerHTML = myFunction(4, 3);
</script>

</body>
</html>
```

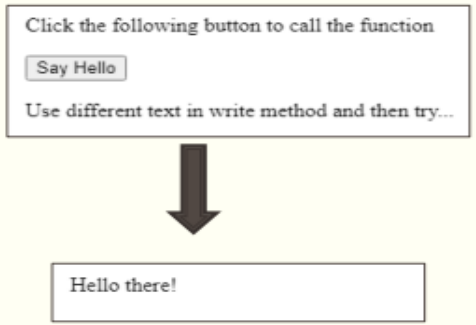
JS Functions

Calling a Function

To invoke a function somewhere later in the script, you would simply need to write the name of that function as shown in the following code.

```
<html>
   <head>
      <script type="text/javascript">
         function sayHello()
         {
            document.write ("Hello there!");
         }
      </script>
   </head>
   <body>
      <p>Click the following button to call the function</p>

      <form>
         <input type="button" onclick="sayHello()" value="Say Hello">
      </form>
      <p>Use different text in write method and then try...</p>
   </body>
</html>
```



JS Events

JavaScript's interaction with HTML is handled through events that occur when the user or the browser manipulates a page.

Example of Events: Web page loads, User clicks a button, Pressing any key, Closing a window, Resizing a window

Events are a part of the Document Object Model (DOM) and every HTML element contains a set of events which can trigger JavaScript Code.

JS Events

On click Event Type

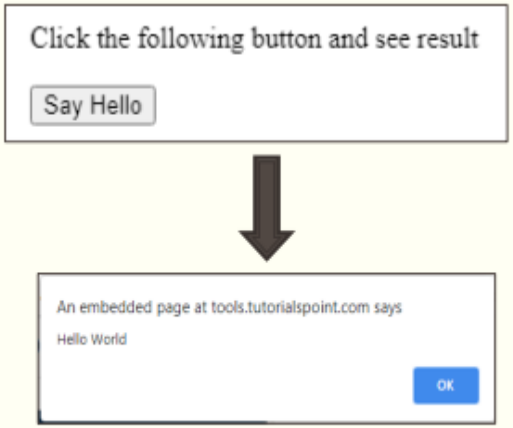
This is the most frequently used event type which occurs when a user clicks the left button of the mouse. You can put your validation, warning etc., against this event type.

```
<html>
   <head>
      <script type = "text/javascript">

            function sayHello() {
               alert("Hello World")
            }

      </script>
   </head>

   <body>
      <p>Click the following button and see result</p>
      <form>
         <input type = "button" onclick = "sayHello()" value = "Say Hello" />
      </form>
   </body>
</html>
```



JS Events

On submit Event type

On submit is an event that occurs when you try to submit a form. You can put your form validation against this event type.

```
<html>
   <head>

      <script type="text/javascript">

            function validation() {
               all validation goes here
               .........
               return either true or false
            }

      </script>

   </head>
   <body>
      <form method="POST" action="t.cgi" onsubmit="return validate()">
         .......
         <input type="submit" value="Submit" />
      </form>
   </body>
</html>
```



JS Events

Onmouseover and Onmouseout

These 2 event types will help you create nice effects with images or even with text as well.

The onmouseover event triggers when you bring your mouse over any element and the onmouseout triggers when you move your mouse out from that element.

```
<html>
   <head>

      <script type = "text/javascript">

            function over() {
               document.write ("Mouse Over");
            }

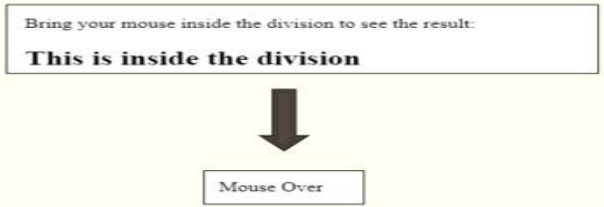
            function out() {
               document.write ("Mouse Out");
            }


      </script>

   </head>
   <body>
      <p>Bring your mouse inside the division to see the result:</p>

      <div onmouseover = "over()" onmouseout = "out()">
         <h2> This is inside the division </h2>
      </div>

   </body>
</html>
```



JS Dialog Boxes

JavaScript supports 2 important types of dialog boxes.

These dialog boxes can be used to raise and alert, or to get confirmation on any input or to have a kind of input from the users.

Types of Dialog Box

- **Alert Dialog Box**
- **Confirmation Dialog Box**

JS Dialog Boxes

Alert Dialog Box

An alert dialog box is mostly used to give a warning message to the users.

For example, if one input field requires to enter some text but the user does not provide any input, then as a part of validation, you can use an alert box to give a warning message.

An alert box can still be used for friendlier messages.

Alert box gives only one button "OK" to select and proceed.

JS Dialog Boxes

Alert Dialog Box

```
<html>
   <head>
      <script type="text/javascript">
            function Warn() {
               alert ("This is a warning message!");
               document.write ("This is a warning message!");
            }
      </script>
   </head>
   <body>
      <p>Click the following button to see the result: </p>
      <form>
         <input type="button" value="Click Me" onclick="Warn();" />
      </form>
   </body>
</html>
```



JS Dialog Boxes

Confirmation Dialog Box

A confirmation dialog box is mostly used to take user's consent on any option.

It displays a dialog box with two buttons: **Cancel and OK**

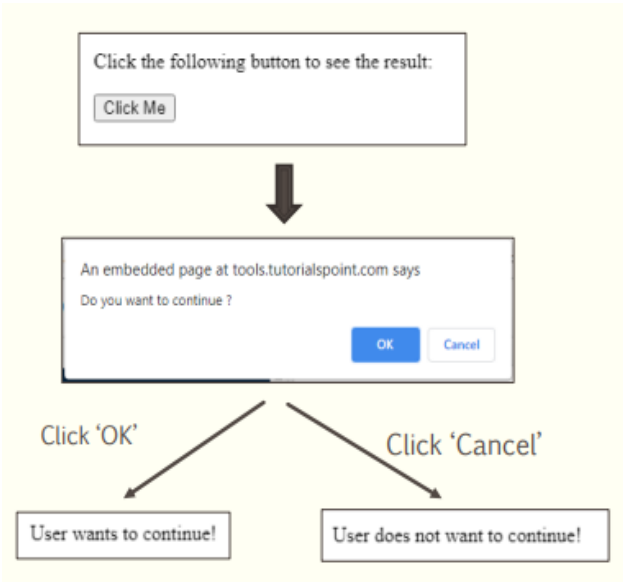
If the user clicks on the OK button, the confirm() will return true.

If the user clicks on the Cancel button, then confirm() returns false.

JS Dialog Boxes

Confirmation Dialog Box

```
<html>
   <head>
      <script type="text/javascript">
            function getConfirmation(){
               var retVal = confirm("Do you want to continue ?");
               if( retVal == true ){
                  document.write ("User wants to continue!");
                  return true;
               }
               else{
                  document.write ("User does not want to continue!");
                  return false;
               }
            }
      </script>
   </head>
   <body>
      <p>Click the following button to see the result: </p>
      <form>
         <input type="button" value="Click Me" onclick="getConfirmation();"
            />
      </form>
   </body>
</html>
```



Thank You...